

SCS2312 - Computational Models and Programming Language Concepts

Take-home Assignment

23000066 - Amarasekara A R M K T

Parser Implementation Report

1. Context-Free Grammar (CFG)

The following grammar represents the simple language implemented in my parser. The language supports integer variable declarations, addition expressions, and print statements.

Program → Statement*

Statement → Declaration | PrintStmt

Declaration → 'int' ID '=' Expression ';'

PrintStmt → 'print' '(' Expression ')' ';'

Expression → Term ('+' Term)*

Term → NUMBER | ID | '(' Expression ')'

Terminals: int, print, ID, NUMBER, =, +, ;, (,)

Non-terminals: Program, Statement, Declaration, PrintStmt, Expression, Term

Start symbol: Program

The grammar is simple and designed only to support addition. Since only '+' is supported, there is no need for multiple precedence levels like in full arithmetic grammars.

2. Token Categories

Before parsing, the source code is converted into tokens. The tokenize() function reads the source character by character and identifies tokens.

INT - keyword 'int'

PRINT - keyword 'print'

ID - variable names

NUMBER - integer values

ASSIGN - '='

PLUS - '+'

SEMICOLON - ';'

LPAREN - '('

RPAREN - ')'

END - end of input

The tokenizer skips whitespace, reads full words for identifiers or keywords, reads full digit sequences for numbers, and handles single-character operators directly.

3. Parser Implementation (Recursive Descent)

I used recursive descent parsing. Each grammar rule is implemented as a function. The parser starts from `parseProgram()` and continues until the `END` token is reached.

Main parser functions:

- `parseProgram()` > parses multiple statements.
- `Statement()` > decides whether it is a declaration or print statement.
- `Declaration()` > handles `'int ID = Expression ;'`
- `parsePrint()` > handles `'print ( Expression ) ;'`
- `Expression()` > handles addition operations.
- `Term()` → handles numbers, variables, and parenthesized expressions.

Helper functions such as `peek()`, `advance()`, and `expect()` are used to control token flow and check syntax correctness.

4. Symbol Table

The symbol table is implemented using a simple array of structures. Each entry stores the variable name and its integer value.

When a variable is declared, it is added to the table. When a variable is used in an expression, the program searches for it using linear search.

If the variable is not found, a semantic error is reported.

5. Error Handling

The program detects both syntax and semantic errors.

Syntax Errors:

- Missing semicolons
- Missing '='
- Invalid expressions
- Unexpected tokens

Semantic Errors:

- Using a variable without declaring it first
- Declaring the same variable more than once

When an error occurs, the program prints an error message and stops execution.

6. Example

Example Input:

```
int y = 5;

int x = 20;

int z = x + y;

print(z);
```

Output:

25

```
PS C:\Users\user\Documents\VS Code\kompile 1.0> gcc -o parser parser.c
PS C:\Users\user\Documents\VS Code\kompile 1.0> ./parser input.txt
25
PS C:\Users\user\Documents\VS Code\kompile 1.0> |
```

This shows that the parser correctly evaluates expressions and prints the result.